

Where Pathfinder Hurts

Shortcomings, a cross-repository failure catalogue, and directions for reducing friction

Pathfinder operator notes

29 September 2026

Abstract

This note consolidates two things: the operator's list of what does not work satisfactorily in Pathfinder, and a read-only post mortem of run logs from four repositories that ran Pathfinder or its ancestors. The post mortem covers about 2,100 receipts in `pathfinder`, 115 in `statarb`, 481 dispatches in `pathfinder-julien-2` and about 1,500 agent calls in `agQSL`. Its central finding is that failures are either not classified at all or classified by grepping noisy `stderr`, so the supervisor and the operator learn about them late or never. The note ends with directions for improving the code and a list of questions to settle. It is meant to open that discussion, not to close it.

Contents

1	Scope and sources	2
2	Root causes in brief	2
3	Shortcomings reported by the operator	3
3.1	Failure monitoring and reporting (R1, R2)	3
3.2	Apex agent (R5)	3
3.3	Visibility into campaign progress (R2)	3
3.4	Context and memory (R3)	3
3.5	Prefix caching (R3)	3
3.6	Code quality and architecture (R4, R7)	3
3.7	The operator loop is not normalised (R2)	4
3.8	Value of repetition	4
4	Failure catalogue	4
4.1	How classification fails today (R1)	4
4.2	Provider side	4
4.3	Launch and runtime environment	5
4.4	Context size	6
4.5	Output contract and parsing	6
4.6	Budget and loops	7
4.7	Error reporting fidelity	7
4.8	Supervision, state and alerting	7
4.9	Verification quality	8
5	Directions for improvement	8
6	Harvest from <code>statarb</code> and <code>agQSL</code>	11
6.1	Engine work to take or adapt	11
6.2	Dropped	12
6.3	Migration	12

7	Refactor specification additions from statarb fixes	13
8	Evidence and state failures from julien-2, 1 October	15
9	Julien-2 ten-pair run, 2 October	16
10	Per-stage routing and PCE editing in the engine, 5 October	17
10.1	Routing	17
10.2	PCE editing	17
11	Decisions taken	18

1 Scope and sources

Scope is detection and visibility. Recovery is deliberately left for later: a failure that is not classified cannot be recovered from reliably.

PF: pathfinder

About 2,100 receipts, from the pilot of 11 September through the recursive extension of 28 September.

SA: statarb arxiv_drip

23 runs, 115 receipts, 23 to 29 September.

J2: pathfinder-julien-2

481 dispatches. Pilot on 25 September; the top-ten batch of 28 and 29 September made 208 charged attempts and ended with all 10 pairs blocked.

AQ: agQSL

Pathfinder3 phases A to E, the Julien edition and phase E prime: about 1,500 agent calls, 1,763 campaign events and 284 phase B failure rows, 20 August to 14 September.

In the tables, *yes* means classified and surfaced automatically, *partial* means recorded but generic, misclassified or found only after the fact, and *no* means invisible without manual digging.

2 Root causes in brief

The friction reported by the operator and the failures in the logs trace back to a short list of structural causes. Each later section refers back to these.

- R1 No error taxonomy.** Provider and runtime failures collapse into a generic error, or are classified by regex over stderr tails that are drowned in noise. Supervisors recognise almost nothing.
- R2 No shared event format or campaign state.** Each component logs and reports health in its own way. Neither the operator nor the apex agent has a single view of where a campaign stands.
- R3 Context is inlined, not referenced.** Ledgers and evidence are pasted into prompts, which overflow input limits, inflate cost and defeat prefix caching.
- R4 Brittle output contracts.** Agent output is checked by large, special-cased parsers and provenance validators that produce both real and false blocks, each fixed by a new patch.
- R5 A supervisor that notices but does not act.** The apex agent knows what to do but does not do it, alerts are lost or repeated, and the supervisor shares a failure domain with the workers.
- R6 Unreliable accounting.** Budgets cannot trip under subscription billing, receipts are double counted, and operational exhaustion is recorded as a scientific outcome.

R7 Lessons do not propagate. The code is not modular enough to share fixes between lineages. AQ already had quota detection, a health policy that retries once and probes with one seat, and a size preflight; SA had its own size preflight and recovery diagnoser; canonical PF has none of these, while it does have a 60 s no-start grace that AQ also built separately. In the same week, main and julien-2 each moved evidence out of prompts, independently and in incompatible ways.

3 Shortcomings reported by the operator

3.1 Failure monitoring and reporting (R1, R2)

Failures are not monitored systematically. New failure modes keep appearing and are discovered by hand, after the fact. The operator named provider censorship (biology questions in the second Julien campaign), token allowance exhaustion, server congestion, agents not talking well to parsers, internal loops such as the research phase running over budget, and process launch troubles that do not bubble up. All of these are confirmed by the post mortem in Section 4, along with many more.

3.2 Apex agent (R5)

The agent trusted with overnight runs does not recover from failures, even when notified and launched with an explicit “be autonomous” prompt. Asked afterwards, it always knows what it should have done. The gap is between diagnosis and action. Candidate causes: model choice, prompt, harness (notifications that do not start a new turn, or a turn that ends waiting for a human), and missing permissions. When it does act, it may act blindly: on 29 September (SA) it restarted a stopped batch without diagnosing the repeated error, and the batch stopped again on the same blocker. The logs show it idle for about 39 hours (PF, 25 to 27 September), about 5 hours (J2, night of 28 September, until the operator typed “fix and resume all”), and 10 hours (AQ, 9 to 10 September).

3.3 Visibility into campaign progress (R2)

There is no normalised way to see which phase or thread is running, what has completed, what is stuck or failed, and how much budget has been spent against plan. Two consumers need that view. The operator currently digs through logs and run directories, differently for each version. The apex agent reconstructs the state by hand each time, which is costly and may be part of why it does not act. The same machine-readable state should serve both.

3.4 Context and memory (R3)

Agents are handed everything inline. On 28 September Vera, the verifier in the EVA loop, received a 9.1 million character request (1.2 million input tokens). There is no way for information to live outside the context and be pulled in on demand, through reading tools or a structured memory the agent can query.

3.5 Prefix caching (R3)

Prompts are not arranged so that stable material forms a common prefix that provider caching can reuse. Repeated calls in loops such as EVA pay full input price.

3.6 Code quality and architecture (R4, R7)

The code has grown by accretion. Parsers are bloated because each agent/parser mismatch adds a patch rather than tightening the output contract. Monitoring is heterogeneous across components. Pathfinder is not modular enough to swap, test or reuse pieces, for example the repetition scheme or the apex agent.

3.7 The operator loop is not normalised (R2)

Working with Pathfinder follows a recurring loop, comparable to a REPL: report on what a run did, recap the state, decide and apply fixes, choose next steps, run again. Each turn of this loop is currently improvised: its inputs are gathered by hand, its output has no fixed shape, and nothing carries one iteration's decisions into the next. The operator wants the loop captured as a skill with a normalised format, so that every iteration looks the same whether the operator or the apex agent runs it.

3.8 Value of repetition

The new version runs EVA three times independently and then synthesises the three threads. Whether this is a good use of budget has not been measured. Open points: whether the threads actually diverge; whether diversity should be designed in (different models, prompts or angles) rather than left to sampling; whether repetition should be adaptive, adding threads only when confidence is low or threads disagree; whether threads should exchange findings midway; and whether the synthesis keeps minority findings.

4 Failure catalogue

4.1 How classification fails today (R1)

- PF records only completed, error, timeout, no session or launch failed plus free text (pathfinder/transport.py:327-335); health.py counts error rows.
- J2 reports nearly everything as Runtime exit N; see retained stderr and classifies only the biology filter, and only in top-ten mode.
- SA's supervisor recognises two signatures and labels the rest unrecognized_operational_failure: 10 of 12 blocked outcomes.
- AQ went furthest but greps the last few thousand characters of stderr, which hold about 2,650 lines of model-list refresh noise plus thread slugs and task text. A slug containing quantum-memory-capacity tripped the capacity rule, and 10 of 16 capacity calls show no capacity wording. Elsewhere in AQ only the exception class or return code survives.

4.2 Provider side

Mode	Signature	Seen	Detected	Notes
Safety refusal	safeguards flagged this message; flagged for possible biological risk; provider_response_rejected	PF 09-11 (3); AQ 09-06 (3 pairs); J2 09-28, Q016 x P011 (2 of 3 branches)	partial	Nondeterministic: identical branches or retries pass. Fires on prompt content before tool use. J2 stop file written by a human. In AQ dependent pairs never started and nobody noticed for 5 days.
Quota exhausted	You've hit your usage limit ... try again at Oct 4th; You've hit your session limit	AQ 09-06 to 09-10 (6); PF 09-19 (41 in 24 s); J2 09-29 (11 calls, 6 pairs in 76 s)	AQ yes; PF, J2 no	No global stop in PF or J2: every queued pair launches and fails. Reset time is in the message but not parsed.
Quota disguised as rate limit	HTTP 429 after a server-directed pause	AQ 08-26 (whole ELM phase A)	partial	Real cause found by hand; ELM spend counter lags, so cost guards read stale spend.
Rate limit	rate_limited	AQ Aug, 09-06 (16)	yes	None in PF, SA, J2.
Capacity, 503, disconnect	Selected model is at capacity; 503 Service Unavailable; stream disconnected before completion	AQ 09-07 to 09-12 (819 lines); PF (6); SA 09-25 (5)	partial	Retried inside the CLI with no host counter. In AQ a 503 tail after a written response made finished calls look failed.

Mode	Signature	Seen	Detected	Notes
Auth failure	401 Unauthorized: Incorrect API key provided; Missing environment variable: ELM_API_KEY	PF 09-19 (40), 09-25; SA 09-25; AQ 09-09	partial	Retried 5 times as transient. No preflight. PF supervisor shared the key and died too.
Key file misconfigured	ApiKeyError	AQ 08-31 (100 in one lane)	partial	Checked per call, not before the batch.
Network and DNS	gaierror	AQ 08-30/31 (172)	partial	Stored as class name only.
Silent model switch	Observed model differs from pinned route; ModelMismatchError	J2 09-25; AQ Aug (6), 09-10	partial	Failed model still billed.
External services	arXiv HTTP 406, HTTP 429; latexpand crash; missing .sty	PF (about 20 pairs); SA intake (2)	yes	No retry or backoff in SA intake.
Market data API timeouts	api.hyperliquid.xyz read timeout (10 s, 45 s)	SA 09-30 (22 blocked steps over 4 h)	partial	API slow, not down (200 in 1.5 to 7.6 s). No retry until fixed on 09-30. Blocks paper-trading steps of live experiments.
Neighbouring services unmonitored	database disk image is malformed every minute; quotes 18,000 s stale	SA Polymarket services, 09-30	no	Noticed only in passing by the recap agent.

4.3 Launch and runtime environment

Mode	Signature	Seen	Detected	Notes
Fast repeated launch failure	transport_process_error, return code 1, 0 bytes, 0.02 s	AQ 09-05/06 (164 in one run)	no	No stderr kept; nothing stopped the retries.
CLI flag drift	unexpected argument '--search' found	PF 09-23 (22); SA 09-24 (2)	partial	SA shows only TransportFailed. PF health probe used other arguments and logged health restored 4 times.
CLI and toolchain drift	Runtime version changed: codex; Codex 0.154.0 hanging in the loader	J2 (2); AQ 09-10 (3 episodes)	J2 yes; AQ no	CLI auto-updated mid-campaign. AQ pinned binary lived in /private/tmp and a reboot wiped it.
Silent hang	no events, timeout, 0 tokens at 900 s	AQ 09-10 (60)	no	About 1 h to diagnose; AQ then added a 60 s no-start timeout. The only true hangs found.
Child segfault	exit -11 before session	PF 09-23 (65 in 60 min)	partial	Probe loop had no deadline. Workaround no_proxy=*
No session in grace	outcome no session	PF 09-19 (241)	partial	Cause not diagnosed.
Sub-agent spawn swallowed	collab spawn failed: no thread with id	SA 09-23/24 (3 of 3 legacy runs)	no	Multi-agent research silently ran single-agent.
Sandbox and tool friction	Operation not permitted; No module named 'pathfinder'; File name too long	J2 (about 2,000 lines); SA (3.3 % of tool commands); PF, AQ (about 20)	no	Agents burn turns on workarounds; exit 0 hides it.
Approval reviewer blocks launch	Automatic approval review rejected launch	J2 09-28	yes	Unattended launch waits on a human.
Lock contention	waiting_for_pipeline_lock	SA 09-28	yes	No timeout for a stale lock.
Background launch dies silently	PID recorded, no process, empty worker log, queue still prepared	SA 09-29 19:3x	no	The launch PID was taken as evidence that reviews were running. Relaunched as a detached session with redirected streams.

Mode	Signature	Seen	Detected	Notes
Nested sandbox blocks the apex agent	pipeline's strategy sandbox raises <code>PermissionError</code> inside the agent sandbox; request to disable the sandbox denied by the permission classifier	SA 09-30	yes	The agent could not resume the batch it supervises and handed a launch command back to the operator.
Misread process state	<code>launchctl nested state = active</code>	AQ 08-21	no	

4.4 Context size

Mode	Signature	Seen	Detected	Notes
Hard input limit	Input exceeds the maximum length of 1048576 characters	SA 09-28 (1, plus 4 just under); J2 (3, up to 9.1 M chars); AQ 09-07	partial	SA and AQ phase B have a preflight; main Pathfinder does not.
Structured data pasted whole	review prompt 1,272,855 chars; one preflight event of 90 cases (605,835 chars of JSON)	SA 09-29 19:54	yes (preflight)	The size gate worked, but the batch stopped. Pooling only deduplicates long strings, not large objects. Fixed by abridging long lists with a count and hash.
Unbounded prompt growth	Vera request 9.1 M chars, 1.2 M tokens; peer prompts up to 91 M chars	J2 09-29; AQ (4 to 5 M tokens)	no	Ledger and evidence inlined. J2 repair 02 routes oversize through read-only packets instead of capping. Cost grows through prompt size, not call count.

4.5 Output contract and parsing

Mode	Signature	Seen	Detected	Notes
Unparseable reply	no JSON object in reply; <code>consolidate: no note;</code> <code>JSONDecodeError;</code> <code>InstrumentError</code>	AQ 08-31 (134); PF (15); J2 (6)	yes	Not distinguished from a scientific outcome. Each slip fixed by a manual patch.
Provenance and binding checks	artefact digest mismatch; complete artifact contradicts inline result; Missing local evidence referenced by; Aliased evidence directory	AQ 09-08 to 09-12 (155 deferred, 106 failed); J2 (blocked about 8 of 10 pairs)	yes, often false	Largest single class. J2 reads equations, ratios and DOIs as paths, fixed three times. AQ: one pair needed 6 passes, each fixing one binding and breaking the next.
Schema-valid, contract-invalid	READY needs code and feeds within the configured allowance	SA 09-29 (9)	partial	Agent proposes feed-free studies; message wrongly suggests an allowance overflow.
Style gate, no repair	Banned style found	SA 09-29	partial	Discards a 6-minute call.
Style gate on factual text	counted framing rule fired on a feed inventory	SA 09-29 (Alpha046)	partial	The rule targets rhetoric but matched a factual list of eight feeds.
Hash basis mismatch	receipt hashes the saved file, reviewer hashes the source string	SA 09-30 (2609.28463v1)	no	One trailing newline paused a study the reviewer itself called coherent. Receipts now carry both hashes and state the rule.
Over-strict candidate check	exact floating-point equality on a normalisation	SA 09-29 (2609.25617v1)	partial	Needed a tolerance of 10^{-12} .
Agent-written patch broken	indentation error in the host patch	SA 09-29	partial	Caught before execution.

Mode	Signature	Seen	Detected	Notes
Non-UTF-8 output	<code>UnicodeDecodeError('utf-8' from latexmk</code>	SA 09-28	yes	Crashes the runner.

4.6 Budget and loops

Mode	Signature	Seen	Detected	Notes
Research allowance exhausted	<code>allowance ran out before every peer declared ready; PAUSE-ON-ITERATE</code>	PF 09-13 to 09-28 (about 15); AQ 09-13	partial	Recorded as a scientific state. PF guard checks only at admission; one run went 6.58 USD over.
Hard budget kill	<code>CEILING REACHED: stopping coordinators</code>	AQ (at USD 1,019)	yes	Left 8 orphaned calls and dead reservations.
Timeout too tight	<code>timeout at 600 s or 300 s</code>	PF 09-11 (2); AQ (1)	yes	Consolidation needed up to 1,344 s.
Budget unenforceable	<code>cost: None</code>	SA (115 of 115)	no	A 60 USD ceiling can never trip.
Metering and double counting	<code>recovery copies receipts.jsonl</code>	PF 09-25; SA 09-29	no	Naive aggregation double counts.
Refusals and cancellations charged	<code>quota refusals hit the attempt cap; Runtime exit -15</code>	AQ; J2 09-28 (4)	partial	Counted against ceilings.

4.7 Error reporting fidelity

Mode	Signature	Seen	Detected	Notes
Message discarded	<code>only exception class or return code</code>	AQ (284 rows and runner events)	no	AQ recovery runs reached r9, suggesting blind retries.
Stored error is noise	<code>last 1,500 chars of stderr; Reconnecting... 2/5 stored as final</code>	AQ (most calls); PF (3)	no	
Cancellation hides cause	<code>BLOCKED: Runtime exit -15</code>	J2 09-28	no	Names the SIGTERM, not the refusal.
Pointer to empty file	<code>see implementation-stderr.log (0 bytes)</code>	SA 09-25	no	Real error only in <code>events.jsonl</code> .
Corrupted JSONL	<code>raw Traceback inside runner-*.jsonl</code>	AQ (9 of 18 files)	no	Breaks consumers.
False success	<code>"finished": true, "error": null with 14 incomplete pairs</code>	PF 09-23	no	
False failure	<code>503 tail after a written response</code>	AQ	no	Needed a salvage flag.
Crash after completion	<code>FileNotFoundError with no path</code>	SA 09-25	partial	
Duplicate incidents	<code>69 records for 7 incidents</code>	PF	n/a	Inflates counts.
Stale error field	<code>READY candidate keeps its preparation-stage error message</code>	SA 09-29	no	Completion changes status without clearing the error.

4.8 Supervision, state and alerting

Mode	Signature	Seen	Detected	Notes
Apex does not act	supervision needs operator; finished_with_blocks; stopped_repeated_operational_failure	PF (39 h); J2 (5 h); SA (2 stops); AQ (10 h)	detected, no action	SA failure counter resets on restart.
Blind restart	agent's own recap: "I should have diagnosed the repeated error before restarting the batch"	SA 09-29 12:30 (second stop on the same READY/feeds blocker)	no	The supervising agent resumed a stopped batch without diagnosing why it stopped, so the same contract mismatch consumed two more investigations. Nine identical errors were treated as nine independent failures. Nothing requires a diagnosis before a restart.
Resume path not idempotent	resume loop raised on rows already stopped and refused to start from a blocked queue	SA 09-30	no	The previous recap had declared that no recurring execution blocker remained; the resume path had never been exercised. A resume must be dry-run before being declared ready.
Admission window expiry	4-hour admission window lapsed during a stop	SA 09-30	yes	Resuming needs a fresh operator approval; the apex agent cannot extend it.
Wrong state, alert fatigue	authentication blocked repeated	AQ 09-09/10 (110 alerts in 10 h)	detected, wrong	12 of 13 blockers were really missing output.
Alert delivery lost	RuntimeError: target unavailable	J2 09-29 (10 to 11)	no	Why the quota cascade went unnoticed.
Stale running status	may be stale after controller exit	J2 (3 branches); PF	partial	No liveness reconciliation.
Stale checkpoint or review	Checkpoint already exists with different content	AQ 09-11 to 09-13	partial	Pre-repair review reused.
Retry discards steering	retry replaced the resumption brief	AQ 09-12	no	Corrective brief lost.
Starvation and slow pipeline	one pair leased per thread; one export per 10 min	AQ 09-10, 09-13	partial	6 h idle threshold.
Retry storms	bursts of identical errors	PF, AQ, J2	no	Up to 196 failures in an hour, no rate alarm.
Shared failure domain	same 401 kills supervisor and workers	PF 09-25	no	Same credentials and transport.

4.9 Verification quality

Mode	Signature	Seen	Detected	Notes
Verifier never rejects	DRAFT, round: 1; verify calls 10 to 28 s, under 550 output tokens	SA (16 of 16 runs)	no	Reject and revise paths never exercised. Possibly legitimate; needs a test.

5 Directions for improvement

These are starting points for discussion, ordered by how much friction each removes per unit of work. Each names the root causes it addresses.

- D1 One event schema and a derived campaign state (R2).** Every component appends typed events to one strict JSONL stream per campaign: call started, completed, failed with class, stage transitions, budget spend. A small reducer turns the stream into a campaign state file. The operator gets a single status command over it; the apex agent reads the same state through a tool. This also fixes stale statuses, duplicate incidents and corrupted logs.

- D2 A shared error classifier** (R1). Classify from the structured events the CLIs already emit (`turn.failed`, `stream JSON`), never from `stderr` tails. Classes: refusal, quota (with parsed reset time), rate or capacity, auth, input too large, launch or argv, no session, timeout, cancelled (with its cause), parse, contract, environment. Each class carries a scope (call, pair, campaign), so quota and auth become campaign-wide conditions. The catalogue's signatures become the test fixtures.
- D3 Preflight before a batch** (R1, R7). Credentials, pinned CLI version and flags, toolchain outside `/private/tmp`, prompt size against the limit. AQ already has most of this.
- D4 Context by reference** (R3). Prompts cite files and agents read them through tools; a size cap raises an event instead of silently packaging. Order prompts as stable prefix then variable tail to benefit from caching.

Status on 29 September: main commit 81f888e moves single-engine consolidation and verification to file tools, and julien-2 has a reviewed counterpart for the joint EVA stage (handover in `plans/2026-09-29-1758-julien-2-joint-evidence-by-reference-handover.md`): a hash-bound index of evidence files, a read-only sandbox for the readers, and reference validation kept. On the retained Q002-P024 joint thread, review material falls from 1.0 M to 0.34 M characters, almost all of it the two papers. The two differ on read-only versus writable tools and on keeping validation, and account, editor and fidelity stages still inline the whole thread. Phase 3a (up to bf8d85e) passes evidence and, above a budget, papers by path, size and sha256, with validation kept and read-only verifiers; session economy is measured on the next live run against the 30 September statarb figures (3.8 to 5.9 M input tokens per paper). Measured on 3 October (pathfinder economy): statarb's 17 investigations on engine-v1.1 took 5.8 M input tokens per pair (median), its first on engine-v1.2.2 took 5.5 M. Context by reference shrank the peers' prompts (8 k characters) but not their tokens: peer calls still take 1.4 to 1.6 M input tokens each, 91 to 94 % cached, because an agent session resends its context on every turn (35 tool calls per peer call on v1.2.2). Consolidation and verification prompts stay near 280 k characters: under the 300 k default budget the papers are still pasted. The levers are a lower `prompt_budgets` for those stages and fewer, more focused peer turns, not the prompt size. One sample after the change; to be repeated. Acted on in engine-v1.3.1: `tool_call_budgets` per role tells each agent how many tool calls to plan for and to read in slices, and pathfinder `health` names stages that overrun; statarb sets 15 for peers and 12 for the editor, and `prompt_budgets` of 60 k characters for consolidation and verification, so their papers go by reference. Measured on 4 October on 2610.01115v1 (engine-v1.3.1) against 2609.39261v1 (v1.2.2): peer calls fell from 1.55 to 0.41 M input tokens and from 35 to 16 tool calls each, and the paper from 5.51 to 4.14 M in all; but consolidation rose from 281 to 572 k and verification from 93 to 223 k, because with the papers by reference they read them back over many turns. statarb therefore dropped the consolidation and verification prompt budgets (3b96fcf) and keeps the tool-call budgets; about 3.7 M per paper is expected. A controlled rerun of 2610.01115v1 the same day (same frozen engine, inputs and tool budgets, no prompt budget) confirmed it: consolidation 210 k (5 tool calls) and verification 69 k (none) against 572 k and 223 k; the paper took 3.12 M in all, with the same verdict (DRAFT in round 1) and a note of the same length, though with 4 peer calls instead of 7. What remains is each peer's first call: in both runs it ignored the budget (21 to 34 tool calls, 0.87 to 1.23 M input tokens) and accounts for about two thirds of the peers' tokens, while follow-up calls stay within it (6 to 18 tool calls, 0.17 to 0.31 M). The cause, read from the receipts' command records: one oversized read early in the call that every later turn resent, the whole paper printed at once (127 k characters) or a search of `supporting.json`, whose lines reach 124 k characters, returning 251 to 370 k; plus side work (compiling LaTeX derivations, running the style gate, one ledger command per entry). Acted on in engine-v1.3.2: peers reading by reference get an outline of each paper by line, a budgeted role is told to bound what it prints, and peers keep their working files as text or code, uncompiled and outside the style gate (which applies to the pipeline's final documents only), and chain ledger commands; statarb (1fe851d) gives the peers their supporting material as a folder with an index and one file per strategy document, without a second copy of the paper. A third run of 2610.01115v1 on engine-v1.3.2 (same inputs and budgets) took 2.61 M input tokens against

3.12 M, same verdict (DRAFT in round 1), a slightly longer note and ledger: the peers' first calls fell to 0.70 and 0.74 M with 14 and 16 tool calls, within budget; they read by line range from the outline from their first command, the largest single output was 51 k characters, and they wrote Markdown and Python, compiling nothing and running no style gate. What remains per call is the turns times the session's fixed context, which no read bound removes. Garbage collection (engine-v1.3.3, 4 October): each finished cycle (research terminal, editing done) removes what its record does not need, namely LaTeX by-products, model-call working directories and the prompt copied into each retained research response, and receipts keep each tool call's command and the length of its output with only its first and last characters; `pathfinder gc` applies the same to finished threads and older receipts, and `"gc": false` turns it off. `julien-2`'s ten-pair campaign carried 4.7 GB of such state (dispatch transcripts, request payloads up to 137 MB, handoff bundles, copied sources) against a 0.57 GB record; it was removed, and `julien-2`'s batch runner now collects each experiment when it completes.

- D5 Contracts over parsers** (R4). One declared schema per stage, validated once, with a single repair turn on violation. Contract failures become an operational class, separate from scientific outcomes. Provenance checks validate declared references only, not paths extracted by regex. Phase 5: `pathfinder/contracts.py` declares the scan, verdict, request review and paper review schemas; a violation gets one tool-less repair turn, and a second violation blocks the pair with failure class `contract` (reconcile reissues a composable request, reruns a single-engine verifier). Declared references only since phase 3b.
- D6 A supervisor that acts** (R5). Wake on events rather than polling; read the campaign state; follow a playbook mapping error class to action; deduplicate alerts and confirm delivery; run on credentials and a transport independent of the workers. Phase 4c (up to a9e0b8a): `pathfinder` `playbook` turns the state into ordered actions with an owner (engine, apex, operator) and a command; repeated alerts are counted, not resent. Waking on events and credentials independent of the workers remain open. Done in phase 9: an audit runs as soon as `events.jsonl` records a stop, a failed call, a unit that blocks or stops, or the run ending (`woken_by` in the session); `"supervisor": {"codex_home", "env", "model", "codex"}` runs it on its own credentials.
- D7 Accounting in tokens** (R6). Meter tokens as the primary unit under subscription billing, deduplicate receipts by call identifier, and charge cancellations separately. Done in phase 9: `"budget": {"calls", "input_tokens", "output_tokens"}` guards admission, projecting the calls in flight at the mean usage so far; every receipt carries a `call_id` and is read once per call; interrupted calls count and are shown apart; `budget_usd` is optional.
- D8 One upstream, thin deployments** (R7). Transport, classifier, event log and state as library modules that `julien-2`, `statarb` and future campaigns import rather than fork. This is where the tidy and simplify sweep belongs. Phase 9: `research.py` (1,150 lines) is split into `thread.py` (what every thread shares), `review\_evidence.py` (the material a composable review reads), `composable\_research.py` and the single-engine thread, with every name still importable from `research`; no unused function was found in the package.
- D9 A skill for the operator loop** (R2, R5). Drafted on 29 September as `repres` (report and resume), shared by Claude Code and Codex from `~/codex/skills/repres`. Codify the report, recap, fixes, next steps cycle as a skill with a fixed output template. Report and recap are generated from the campaign state of D1 and the failure classes of D2, not assembled by hand; fixes and next steps are recorded as structured entries that the next iteration reads back. The same skill serves the operator interactively and the apex agent overnight, which also gives the apex agent an explicit procedure to follow when something fails.

A first template, extracted from a typical recap the operator received on 29 September (SA, 13:11 Paris), with gaps in that recap added:

1. *Status* with timestamp: running or stopped, since when, and why, in one sentence; whether any agent is running.

2. *Totals* as fixed counters from the campaign state, with the change since the previous iteration (the example gave totals but no delta).
3. *Blockers*, grouped by failure class with counts, so that nine instances of one cause read as one blocker.
4. *Corrections*: what the previous iteration got wrong.
5. *New results* since the previous iteration.
6. *Still active*: live items with their health measures.
7. *Fixes applied* in this iteration (missing from the example).
8. *Fixes to apply next*: repairs to code, contracts or artefacts that are diagnosed but not yet done, each tied to the blocker it removes.
9. *Next steps*: campaign actions (resume, relaunch, pause), ordered, each with a precondition to check before acting; a restart requires a diagnosed blocker and its fix applied.
10. *Decisions needed from the operator* (missing from the example).

D10 Measure before redesigning EVA. Measure divergence across the three EVA threads, and test Vera by planting flawed notes to see whether it rejects them.

A suggested order: D2 and D1 together first, since they are the detection foundation and fit the current scope; then D3 and D4, which are cheap and remove whole failure families; then D5 and D6; with D8 as the frame for the sweep.

6 Harvest from statarb and agQSL

Read-only exploration on 29 September of statarb's `arxiv_drip` and of agQSL's latest generation (phase E prime with the Julien transport and the pathfinder3 peer coordinator, last code commit 14 September; older generations ignored), each compared with canonical `pathfinder` at 81f888e.

6.1 Engine work to take or adapt

Item	Mechanism	Source	Canonical today, and what to do
H1	Failure classes from structured events: quota, auth, rate, with deployment-registerable rules	AQ QuotaPaused; SA recovery diagnoser	Only generic outcomes. Classify from <code>turn.failed</code> payloads, never <code>stderr</code> . Quota stops admission with reason <code>quota</code> and does not BLOCK the thread; auth stops; 429 defers. (D2) Done in phase 1 (commits 77da501 to 20f283e): classes, scopes, reset time, campaign stops, deployment rules; 429 deferral done in phase 4a as a campaign cooldown at admission, with one retry of unserved calls.
H2	Prompt size gate before admission, refusal receipt, lossless JSON compaction, pooling of large sources	SA <code>prompt_context.py</code> ; AQ input hygiene	Only a word count at the edit stage. Take into <code>transport.execute</code> . (D3, D4) Size gate done in phase 1 (4b1f9b9); compaction and pooling in phase 3.
H3	Health policy: retry once, flag after two consecutive failures, auto-clear, single-seat probe	AQ runner	Any stage exception drains the run; restart is manual. Adapt. (D6) Done in phase 4a (up to 2dee0d9).
H4	Per-call last-event time	AQ runner	<code>health.snapshot</code> has heartbeat and overdue calls but no event age. Stamp it in <code>active-calls/*.json</code> . (D1) Done in phase 2a as the event stream (<code>call_started</code> , <code>call_finished</code> with tokens and tool calls) up to 49ba31f.
H5	Coordinator batch mode: deadline, maximum units, stop marker, opt-in tolerance of k consecutive failures, recovery first, deployment <code>next_unit()</code>	SA <code>overnight.py</code>	<code>coordinator.py</code> runs a fixed schedule and stops on the first failure. Adapt, opt-in. Done in phase 4b (up to 9323be9) without the deployment's <code>next_unit</code> callable, which moves to phase 7.

Item	Mechanism	Source	Canonical today, and what to do
H6	Recovery: one admission per attempt; reconcile actions to collect retained verify output, retry a stage refused before any turn, and mark terminal failure	SA <code>recovery.py</code> ; AQ <code>reconcile.py</code>	<code>reconcile.py</code> has four actions, <code>recovery.py</code> only repairs verdict JSON. Adapt. Phase 4a: reconcile unblocks a repaired evidence block with a recorded transition (up to 2dee0d9); one-admission recovery receipts remain for 4c. Phase 4c: every reconcile action recorded in the stage status history (up to a9e0b8a).
H7	Cross-process seat pool keyed by account, reservations bound to PIDs, dead ones reaped	AQ <code>coordination.py</code>	Admission counts within one process. Needed before statarb and agQSL share one subscription.
H8	Schema-constrained single calls	SA (<code>codex exec --output-schema</code>)	<code>transport.call</code> has no schema support. Add; statarb's scan, implementation and review then use the engine. Done in phase 5: every structured request carries its contract; Codex gets the strict form through <code>-output-schema</code> when "codex": <code>{"output_schema": true}</code> .
H9	Post-edit consumer stage, which cannot overturn the verifier	SA implementation stage	No such hook. Adapt as an extension point. Done in phase 5: the consumer extension runs after an edit, is recorded in <code>consumer.json</code> , and any change it makes to the research or edit status is undone.
H10	Monitor: Host and Origin checks, CSP, file allowlist and size cap; aggregate across campaigns; per-stage unit counts	SA watchboard	<code>monitor.state()</code> already has parameters, scores, blocks, stages and PDF links, and is the base for the operator page; its server serves any file under the root. Done in phase 2b (<code>pathfinder</code> view, hardened <code>serve</code> , up to e78c8db) and the view across the arms of a coordination done in phase 4b (up to 9323be9).
H11	Incremental intake: watermark advanced on success, dated negative cache	SA intake	<code>corpus.fetch</code> makes one request with no retry.
H12	Test that a repair survives a transport failure during consolidation	AQ	Behaviour already correct in <code>research.py</code> ; add the contract test.

Later candidates: an append-only event log with replay validation (`pathfinder3`, an older AQ generation; aligns with D1) and AQ's full-text acquisition as a corpus adapter.

6.2 Dropped

statarb's engine compatibility shim (its fix is already at HEAD), the deploy hold by monkeypatching, two one-off scripts, copy-then-driver recovery and its direct model calls once H8 lands. From agQSL: the external cost guard (harmful: it once killed the coordinator and orphaned 8 calls), the `stderr` regex classifier, salvage-transport and the Julien watcher (superseded at HEAD), and the ELM route pin (canonical has the backend).

6.3 Migration

statarb. Stays in statarb as deployment code: intake policy, scanner and memory, mandate and briefs, the READY and feeds contract, review, deployment bundle, paper trading and its scheduling, data access, and the watchboard panels as `snapshot_extra`. Steps: its owner commits the pending `arxiv_drip/` work; land H2, H6, H10 and the classifier; add H8 and H9 with a deployment test on the stub backend; add H5; tag `engine-v1.2` and repin; replace `overnight.py` with a batch schedule; retire the watchboard server once the operator page exists. Risks: archived campaigns on frozen v1.1 engines; verdict drift from 81f888e while statarb sets inline evidence, to be measured before repinning; "paper" means paper trading in statarb and the manuscript in the engine; two lock regimes during the transition; hard-coded host paths. Done in phase 7 (statarb c0c236c, engine-v1.2.1): statarb pins engine-v1.2.1, makes its own scan, implementation and review calls through the engine with its own schemas (contracts with one repair turn, receipts with failure classes, stop markers and cooldowns) on the shared codex-subscription seats (H7), and builds repair notes on the frozen engine instead of a moving checkout. A live smoke call passed. Not done: `overnight.py` still selects units itself, because statarb runs one campaign per paper while the coordinator's arms are fixed at load (`next_unit`

exists for deployments whose units live in fixed arms). Ruling: `overnight.py` stays, since each `statarb` investigation freezes its own engine and runs in its own worker, which the coordinator does not do. The watchboard's content is available as panels of the engine's operator page (engine-v1.2.3 extension panels; `python -m arxiv_drip view`); the old server is retired once the operator has compared the two; the engine compatibility shim stays for archived investigations frozen at engine-v1.0 and v1.1, where it is still needed and is harmless on later engines.

agQSL. Becomes a campaign repository pinning a frozen engine release. Domain content stays: question and paper sets, papers, prompt overlays, styles, notes and published supplements. Steps: tag the current head `pre-pathfinder-roof`; land H1, H3, H4, H6 and H7; map the ITERATE/PAUSE/DRAFT decision words through a prompt overlay; import only the open pairs into canonical thread directories; pilot two pairs; add agQSL to `deployments.toml`. Archive, never delete: tracked production artefacts stay reachable through the tag; untracked build output and the full-text cache (3.7 GB) go to a compressed archive with a checksum manifest. Risks: ledger schema translation, paths cited by published supplements (leave redirect notes, do not move), the ELM key held outside the repository, and account oversubscription until H7 lands. Done in phase 7: tag `pre-pathfinder-roof` pushed; the five open pairs imported into a canonical campaign in `canon/` (ledger translated: epoch time to ISO at, Ada/Emmy to lowercase, peer directories renamed, each pair's `question.md` kept as an input with its exclusions enforced by an overlay and `peer_search` off; decision words mapped in the verify overlay); engine-v1.2.1 frozen in `canon/engine`; two pairs piloted live: Q1P1 (ART 110) ended research PAUSE with its readable note edited; Q2P2 (ART 37) ended research PAUSE and its edit retry is stopped by a Codex CLI crash (O02). Untracked bulk removed rather than archived, at the operator's instruction (build output, probe outputs, full-text cache, typeset output and an exported package); prunable worktrees and merged feature branches removed, four unmerged ones kept.

7 Refactor specification additions from `statarb` fixes

Fixes applied to `statarb` on 30 September (commit 4466744 there) that generalise to the canonical engine and belong in the refactor specification.

- S1 Context by reference, lossless and budgeted** (D4, H2). Pool repeated strings; write every document over 16k characters, the largest others beyond a 64k inline budget, and every JSON subtree over 16k characters to `context/<sha256>.txt` or `.json`; list them in an index with size and digest; resolve nested references. The reading stage gets a read-only shell. A test reconstructs the full value from prompt and files. Effect on a real handoff: 1.27 M characters to 83 k.
- S2 Agent sessions multiply their prompt.** A peer session re-reads its context on every turn: one peer call recorded 2.4 M input tokens, 94 % cached, for a prompt that inlined a 335 kB paper. Inline flags for papers and ledger must default to off, and receipts should record prompt size, turns and cached share so the amplification is visible.
- S3 A sanctioned launcher that verifies liveness.** Starting a batch goes through one command that detaches it in its own session, keeps one PID across shell handover (`exec`), waits, and fails loudly with the log tail if the process died. No hand-written `nohup` recipes. This is also the route to allow for the apex agent, whose own sandbox otherwise prevents it from launching pipelines that need their own sandbox. Done in phase 4b as `pathfinder launch` (up to 9323be9); the rate-limit cooldown is shared with the coordinator parent and shown.
- S4 Watch by PID and polling.** Name-based process checks break when a shell execs its last command; following a log with `tail -F` can deliver nothing. Supervisors poll line counts and check the PID.
- S5 State transitions clear stale errors.** Every status change writes the error field explicitly, so a completed unit never shows an old failure.
- S6 Derived stores heal themselves.** A cache that fails to open is moved aside with a timestamp and rebuilt; the failure is logged once, not every minute for two weeks.

- S7 Controlled clocks in tests.** Anything gated on wall-clock windows is tested with an injected time on both sides of the window.
- S8 Freshness alarms on external feeds.** A five-hour network degradation on 30 September stalled Hyperliquid collection and Polymarket quotes at once; both recovered unnoticed. Each feed needs a staleness threshold that raises an event (D1) rather than a line in a log.
- S9 Multi-source intake (H11).** Sources beyond arXiv need per-source identifier namespaces, per-source watermarks, deduplication across sources by title and first author with aliases, and an explicit abstract-only status. SSRN qualifies only as metadata through Crossref (prefix 10.2139): its terms forbid automated access and full-text storage, so research prompts must forbid fetching from it. NBER and RePEc NEP feeds and further arXiv categories with a keyword filter are straightforward. Decision (3 October): sources are configured per deployment, never engine-wide. The engine provides source adapters and the shared machinery (namespaces, watermarks, deduplication, abstract-only status); each deployment's `campaign.json` names the sources it uses and their settings. SSRN suits statarb's quantitative-finance intake and makes no sense for proofTree.
- S10 The operator view reads documents, not sources.** Learned from the statarb watchboard on 1 October. A unit's produced documents open as their PDF in a reader inside the page, with the $\text{\LaTeX}$ source one click away and a visible warning when the PDF is older than its source; a unit without a PDF falls back to its source text. Every unit's document has a stable link (`#note=<unit>`) so a PDF can be cited in a message. The page opens with the pipeline itself: one count per stage, split by state, each state a filter on the unit list. PDFs are served inline and may be framed only by the view's own origin. Redesigned in phase 2c after the statarb watchboard and checked by the operator on 1 October: one pipeline position per unit decided by the server, queue, research desk, dialog reader, usage in human units.
- S11 Agent sandbox environment, seen in a third deployment** (proofTree, 1 October; also J2 and SA). Peers call the ledger helper as `python -m pathfinder.ledger`, which fails inside the agent sandbox because the editable install points outside it (20 occurrences in one run); here-documents fail because the shell's temporary directory is not writable; a TeX lookup is denied. Agents recover by improvising, which costs turns and hides the problem. The engine must give each agent workspace a self-contained ledger helper (a script in the thread, not an import), a workspace-local `TMPDIR` and `TMPPREFIX`, read access to the TeX installation with `TEXINPUTS` preserved and local TeX caches, and must count tool errors per call in the receipt. Done in phase 3a (up to bf8d85e): workspace ledger helper, local `TMPDIR`, `TMPPREFIX` and `TEXMFVAR`, `tool_errors` in receipts; TeX read access remains a deployment sandbox matter. Deployment notes from the phase 3a review: deployments that track thread directories in git should ignore `**/threads/**/.pathfinder/`; julien-2 dispatches through its own eva2 transport, so neither reads nor the workspace environment reaches its agents until it adopts the engine transport; a read-only Codex verifier cannot use here-documents. Single-engine peers still receive both papers inline when a campaign sets `inline_papers`, which statarb no longer does.
- S12 Allowances calibrated from observed durations.** proofTree lost two peer calls at a 300 s allowance (one peer had already declared readiness) and a joint editor at 240 s; the fixes were 900 s and 600 s by hand. The engine should warn when a stage allowance is below the durations its receipts have recorded for that stage, and a timed-out session's saved files stay the starting point of the retry. Done in phase 4a (up to 2dee0d9).
- S13 Stopped statuses carry their failure class.** A timeout still reaches the stage status as "transport failed"; the receipt knows it was a timeout. The structured failure recorded for blocks in phase 2a should also be written on stopped statuses. Done in phase 4a (up to 2dee0d9).

8 Evidence and state failures from julien-2, 1 October

Source: `pathfinder-julien-2/reviews/pathfinder-failure-modes-2026-10-01T002939+0200.md`, an inventory of the exploratory pass of 29 September (ten pairs: four DRAFT and one PAUSE in joint research, three controller exits 0 and seven exits 2). Most belong to the provenance family of the catalogue; together they argue for one typed evidence inventory and resolver shared by every stage, instead of more exception strings.

Item	Failure	Refactor target and regression
F01	A citation loses its branch namespace at the joint boundary (<code>ada/repository-audit.json</code> lives under <code>branches/branch-2/</code>)	Stable artefact identity with origin namespace through export and joint citation; test exact, unique-branch and conflicting-branch matches; ambiguity stays explicit. Done in phase 3b: unique bundle match resolves, two matches are an explicit ambiguity. (up to 5203d1c)
F02	Campaign source paths (<code>sources/P072/raw.json</code>) leak into thread evidence	Source registry resolving source and manifestation identifiers; a thread moved to another root keeps its citations; no silent raw-to-text substitution. Done in phase 3b: campaign files named by the pair's corpus rows resolve by their own bytes. (up to 5203d1c)
F03	A code string fragment (<code>.raw.txt</code> suffix) blocks collection as a required file	Explicit declared dependencies, never strings found in executable source; never execute scripts to find dependencies. Phase 3b: the canonical engine reads citations from ledger text only, never from code; the fragment came from julien-2's own collector. (up to 5203d1c)
F04	A generated output name (<code>coverage-and-checks.json</code>) is taken as a consumed input	Declared inputs, planned outputs and observed outputs are distinct; a claim resting on an absent output stays an evidence gap. Done in phase 3b: declaration status output; a missing planned output is a gap. (up to 5203d1c)
F05	A page locator (<code>prose/p.1653</code>) is parsed as a path	Typed citations with document, page and section fields. Done in phase 3b: locators, numbers, DOIs, URLs and repositories are typed, not files. (up to 5203d1c)
F06	External repository paths (<code>chemle/emle-engine.git</code>) are treated as local evidence	Retained local bytes, external citations, unavailable external assets and generated artefacts are distinct kinds; an external citation cannot support a claim of inspection without retained evidence. Done in phase 3b for typing; retained local, external and unavailable stay declaration statuses. (up to 5203d1c)
F07	Repairing evidence leaves a stale terminal BLOCKED status	Explicit reconciliation: verify the repaired precondition, record the old state, keep retained responses and charges, resume at the right checkpoint, once. Done in phase 4a (up to 2dee0d9).
F08	Research accepts (DRAFT) and delivery then fails on a different evidence contract	One evidence manifest and resolver for research, verification and delivery; validate the next stage's contract before spending on it; packaging failure keeps the verdict and resumes without rerunning Vera. Phase 3b: one evidence manifest per review (<code>evidence-manifest.json</code>); validation before delivery belongs to julien-2's collector and moves with its migration. (up to 5203d1c)
F09	A truncated monitoring summary is reported as progress	Bounded structured events; truncation means unknown progress until the full record is read. Done in phase 2a: a cut event stream gives progress unknown.
F10	One finished/blocked label mixes research and delivery state	Separate research decision, evidence readiness, editorial, assessment and controller states. Done in phase 2a: research, editorial, assessment and controller states in <code>state.json</code> .
R01	Repair helper appends <i>unavailable</i> declarations until collection passes	Repairs propose classifications with evidence; alias, non-dependency and genuinely unavailable stay distinct. Phase 3b: <code>pathfinder evidence</code> proposes declarations per error, writing nothing. (up to 5203d1c)
R02	Declaration repair is incremental and rescans everything per item	Build the whole repair from one inventory, validate on scratch data, commit atomically with before and after hashes; idempotent. Done in phase 4c: <code>pathfinder evidence PAIR -apply</code> .
R03	Recovery parses exception messages	Typed error records with origin, document, citation, phase and recovery policy; simultaneous errors stay separate. Done in phase 3b: typed errors, all reported together. (up to 5203d1c)
R04	Pass records name only the Git head while the working tree differs	An execution identifier covering uncommitted code and configuration, without copying the engine per experiment. Done in phase 2a: <code>execution_id</code> in <code>run.json</code> and <code>run</code> events.

Placement in the roadmap: F01 to F06, F08, R01 and R03 define phase 3 (typed evidence inventory and resolver, together with context by reference); F07 and R02 go to phase 4 (reconciliation transitions); F09, F10 and R04 go to phase 2 (event log, separated states, execution identity). Phase 1's typed failure records are the model for the typed evidence errors of R03.

9 Julien-2 ten-pair run, 2 October

Reported from the julien-2 campaign's run of ten pairs. The first is a recurrence: the by-reference fix had covered joint research only.

Item	Failure	Refactor target
J01	Oversized prompts again, now in PCE editing and Claude assessment	One context builder with a prompt budget for every stage (research, consolidation, verification, editing, authoring, assessment), passing large material by reference; tool-less stages get a budgeted digest, never an unbounded paste. Done in phase 5: <code>pathfinder/context.py</code> , budgets in <code>prompt_budgets</code> , a <code>context_built</code> event per prompt.
J02	A local size rejection consumes an attempt although no model call starts	A call refused before launch (size gate, admission stop) never counts against attempts, allowances or ceilings; canonical receipts already mark it <code>refused</code> and uncharged, the attempt counters must follow. Done in phase 4c: composable research treats a refusal as a stop; regression tests.
J03	Account recovery changes the prompt under an existing attempt identifier, so the hash check blocks reuse of completed responses	Attempt identity is immutable: a recovery that changes the prompt opens a new attempt linked to the old one, and completed responses are reused by their recorded request, not by position. Done in phase 4c: reconcile supersedes and reissues with a recorded lineage.
J04	Monitoring reports activity after the controller exits because <code>pgrep</code> loops match themselves	Liveness comes from the runner record (PID, heartbeat) and active-call records, never from process-name patterns (S items on watching by PID). Already the rule in <code>state.json</code> since phase 2a; to be the rule in every supervisor script.
J05	A PCE author call timed out and needs explicit reconciliation	Reconcile covers every stage, not only research: a stopped edit, author or review step names its failure class (phase 4a) and its one safe action. Done in phase 4c: reconcile reruns a stopped edit or paper, resumes the review only after the round's author call returned, and names a blocked edit for the operator.
J06	An assessor's reply was fenced JSON followed by a sentence of prose, and the strict parser rejected it (Q024-P011, Q024-P036)	The canonical reader takes the first fenced block that holds an object, else the first object in the text, and ignores prose around it (<code>json.JSONDecoder.raw_decode</code> from each opening brace). Done in engine-v1.2.2.
J07	A PCE editor call hit its 600 s limit while still working (Q002-P036); the operator retried it with twice the time	A stage whose previous call timed out is rerun once with twice its allowance (<code>transport.extended</code>), recorded as <code>allowance_seconds</code> ; the doubling is not compounded. Done in engine-v1.2.2 for the editor, the author and the paper reviewer.
O01	Agents searching arXiv meet its rate limit ("Rate exceeded.", HTTP 429), often inside a command that exits 0, so no failure is recorded	Read every tool output, not only failed ones, for each source's rate-limit answer; count the hits per call in the receipt (<code>source_limits</code>), warn in <code>health</code> over the last 50 calls, give the apex agent an action, and tell peers the arXiv etiquette (API endpoint, 3 s spacing, one retry after 30 s, record searches not run). Reported by the operator on 2 October: in the proofTree sphere-packing campaign arXiv returned HTTP 429 on queries 14 to 18 of a burst. Done in phase 7a.
O02	Codex CLI 0.160 exits with signal 11 within 0.1 s of launch, before any session, on the agQSL Q2P2 edit retry: six engine attempts in a row, while the same command and prompt run from a shell or a plain subprocess succeed	Classified as <code>launch</code> ; three in a row stop the campaign, as designed. Cause: the edit stage had just checked references on arXiv through <code>urllib</code> 's default opener over <code>http</code> , which consults the macOS system proxy configuration; after that, every child the process forks dies with signal 11. Reproduced in isolation. Fixed in engine-v1.2.2: the engine's downloads use an opener with environment proxies only (<code>pathfinder/net.py</code>) and the arXiv API over <code>https</code> .

Item	Failure	Refactor target
O03	A deployment's evidence scanner mined paths from the researchers' TeX compile logs (ada/compile.out): wrapped font paths and font-shape names (T1/cmr/m/n/10.95) counted as missing evidence and stopped a composition twice; the operator's pane declared 84 exceptions by hand (2 October)	Canon reads cited paths from ledger text and declarations only, never from files, so logs are not mined. Its classifier now also refuses TeX installation tokens quoted in a ledger: font and encoding files, texmf trees, NFSS font shapes (class tex); a researcher's own files keep the strict check. Done after engine-v1.2.2.

The common lesson: prompt budgeting and recovery must be uniform across stages, not fixed stage by stage.

10 Per-stage routing and PCE editing in the engine, 5 October

Two parts of julien-2 still lived in its controller (eva2 and julien2) rather than in the engine: the choice of runtime, model and effort per stage, and PCE editing. Both are now campaign options of pathfinder, so that julien-2 can later become a plain campaign (campaign.json, prompt overlays and its record) with no engine or controller code of its own. Without the new keys every deployment runs exactly as before.

10.1 Routing

julien-2 named two routes in each experiment.json (research on Codex gpt-6-astra, effort medium; an assessor on Claude claude-opus-5, effort medium) and sent its assessment stages to the assessor through stage_routes. The engine takes the same choice as one key:

```
"routes": {
  "peer": {
    "backend": "codex", "model": "gpt-6-astra",
    "effort": "medium"},
  "edit/pce-critic": {"backend": "claude", "model": "claude-opus-5"}}
```

- A key is a stage (scan, peer, consolidate, verify, ledger_review, edit, author, review) or a stage and actor (peer/ada, edit/pce-critic); the stage-and-actor key wins.
- A route names any of backend (codex, claude, elm, stub), model and effort (minimal to max); what it leaves out stays as the stage chose it. Effort reaches Codex as model_reasoning_effort and Claude as -effort.
- Routing happens once, in transport.execute, on a copy of the campaign, so every stage that builds a request is routed alike and none can bypass it. The backend switches per call; admission, receipts and pricing see the routed backend and model.
- A routed call's receipt carries route (key, backend, model, effort); unrouted receipts are unchanged.
- Malformed routes fail at load, the branch and joint overrides of a composable campaign included (pathfinder/routing.py, tests/test_routing.py).

10.2 PCE editing

"edit_scheme": "pce" (default single) makes the single editor's note the frozen baseline of a PCE round, ported from julien2/pce.py over the wwaites/pce role contracts (pathfinder/pce.py). Each pass: the author revises the baseline and lists its material claims; the host copies the draft into the revision history with a custody record and the archivist writes the provenance note; the fact-checker classifies every claim against the external sources only (the papers as the thread read them, and a research supplement made of the thread's ledger, its branch ledgers and the verifier's reasons, which julien-2 also treated as approved generated evidence); the critic reviews as a reader, without internal sources or earlier reviews; once both gates pass, the editor accepts or asks for a revision. An unsupported

claim or a critic's revise sends the next pass back to the author with the history. A contaminated gate, an editor's request for new evidence, a reply that breaks its contract or an exhausted allowance ends the round as `review_required`.

- The contracts are engine prompts, `prompts/pce-<role>.md` with the brief in `pce-brief.md`, so a campaign extends them with append overlays. Each role sees exactly the files its Read Scope lists.
- Calls are tool-free, stage edit, actor `pce-<role>`: routes, admission and receipts treat them like any call.
- The round lives in `edited/pce/`: the input manifest with its fingerprint, the frozen sources, every dispatch with its prompt digest, drafts, claims, reviews, revision history and `result.json`. A retained dispatch is replayed, never paid for twice, so a stopped round resumes where it stopped; inputs or prompts that changed under a begun round block the edit for the operator.
- An accepted draft that builds with clean references becomes `edited/note.tex` and `note.pdf`; otherwise the baseline stays the readable note. Either way the edit is done, and `edit.json` records `editorial_status` (accepted or `review_required`), the reason, and any presentation error.
- Settings under `pce`: passes (1 to 5, default 2), `fact_checks`, `critic` (profile and remit; julien-2 used `computational-chemist`) and `supplement_chars`; `pce_seconds` in the allowances.

The specification `features/operational/edit-stage.feature` described delegating to an installed PCE runner through Nix; it now specifies this port, and its `@pce` scenarios run in the broad suite on the stub backend.

Not ported, with the reason:

- julien-2's recovery grants and imported retained responses: one-off repairs of its own rounds; the engine resumes a round from its own dispatch records.
- The reviewers' separate source projection and the length-framed editor packet: size measures for julien-2's bundles. Here the supplement is capped, and a role prompt over `max_prompt_chars` is refused before launch and ends the round for review. PCE prompts do not yet go through the context builder's `prompt_budgets`.
- The body-only LaTeX allowlist and copyedit: the engine's note is a whole document, checked by its build and reference checks.
- Assessor stages (actionability, fidelity, reader). PCE acceptance does not depend on them: `eva2` assessed actionability after editing, from the research evidence, and compared accounts for fidelity and reader only once a PCE account was accepted. They remain the open part of julien-2's controller; routes will apply to them as soon as they are engine stages.
- julien-2's per-stage dispatch limits: the engine's budget is in calls and tokens for the campaign.

11 Decisions taken

Settled by the operator on 29 September.

1. **Canon.** `pathfinder` is the canonical engine. `pathfinder-julien-2` is treated as a separate branch from which innovations are taken selectively (for example the read-only readers and hash-bound evidence index of its joint-reference candidate); it is not merged wholesale. This settles the frame for D8.
2. **Apex agent.** A strong Codex or Claude agent. It receives failure notifications, handles them, and resumes the campaign itself for most failure classes. Some classes are outside its remit: a usage limit is the reference case, where the right action is to pause until the reset and tell the operator, not to retry. The rule is to handle everything the agent can handle. It escalates only when the resolution lies outside its reach: a usage limit (wait for the reset), credentials it does not hold, or a decision that belongs to the operator, such as changing a research contract or scope. D6 and the `repres` skill follow this rule.
3. **Billing.** Subscription. Dollar cost is not a usable budget unit; budgets and accounting are in tokens and calls (D7), and usage limits are the binding constraint to detect and forecast.

4. **Operator view.** A web page, rendered from the campaign state of D1. The apex agent reads the same state directly, not the page. The page shows:
 - the campaign parameters (models, seats, rounds, allowances, EVA scheme and branch count, budget in tokens and calls);
 - progress so far: for each phase or stage (scan, research, edit, review and so on), the number of units of the batch, typically pairs, that are queued, running, done or blocked there;
 - blocks, grouped by failure class and cause;
 - a list of units with their scores, current stage and status, and links to the PDFs they produced, if any.
5. **Formats.** Logs and receipts need not stay compatible with earlier runs. D1 and D2 may define new formats freely, with no converter.
6. **Operator loop.** Every iteration contains the ten sections of the repres template as it stands, one file per iteration in operator-log/.
7. **Composable EVA in canon.** The julien-2 scheme (independent EVA-minus branches followed by a joint EVA thread over their bundles) is incorporated into pathfinder as an option, not as the default path. The number of branches is a campaign parameter with default 3. The measurements of D10 (thread divergence, Vera rejection rate) then inform the default rather than the existence of the option. Naming (2 October): EVA-minus is called *direct EVA*, the variant in which E writes no synthesis and V reads the ledger directly. The option is then N direct-EVA branches followed by one joint EVA thread; the engine's research_scheme value eva_minus is renamed accordingly in phase 6. Done in phase 6: "research_scheme": "composable" with "branches": N (default 3) and optional branch and joint overrides runs every pair as N direct-EVA branches inside the engine (branch-runs/<label>/), freezes each handoff as a read-only bundle with a sha256 inventory (branches/<label>/bundle.json), and then runs the joint EVA thread in the pair's own directory under the ordinary runner, edit and paper. The joint thread starts only once every branch has handed off; a blocked branch blocks the pair and reconcile repairs that branch. branches/metrics.json records the D10 measurements: Vera's rejection rate per branch and overall, and thread divergence (mean pairwise Jaccard distance of word 3-grams over the branches' substantive ledger text). D10, second half, measured on 3 October (pathfinder probe-vera on agQSL's five finished pairs, one direct-EVA review each of a sound and a planted-flaw copy, one quantity multiplied by three in one finding): Vera asked for more on 4 of 5 flawed and 3 of 5 sound copies, and corrected the planted value, naming both numbers, in 2 of 5 (a stated numeric result and a script's reported check count). The three misses were symbolic factors inside formulas. A review that asks for more is therefore no evidence of a flaw found; the correction rate is the measure, and two of five is low for one-review audits of long ledgers. The value eva_minus is still read, as direct_eva with a deprecation warning, because proofTree writes it.
8. **julien-2's driver** (phase 9). pathfinder import-eva2 EXPERIMENT -out DIR turns an eva2 experiment (three direct-EVA branch campaigns and a joint campaign) into one canonical composable pair: the joint thread with eva2's exact branch bundles under canonical inventories, the branch threads kept as branch-runs/, the settings as branch and joint overrides, the dollar settings dropped. Checked on eva-top-ten-01-q002-p024 (joint DRAFT, bundles verified); the imported pair goes on to edit and paper in the engine. Its 38 calls, read from eva2's runtime store, took 56 M input tokens for one pair (18 peer calls at 3 M each), about ten times a single-thread statarb pair: the cost of three branches and a joint thread, which D10's measurements should be weighed against.
9. **julien-2 cherry-picks** (phase 7, engine-v1.2.3). Taken: the branch and joint briefs as engine role briefs (prompts/branch.md, prompts/joint.md, appended for peers, consolidation and verification; any thread over bundles is a joint thread); the handoff checks a frozen bundle lacked (contiguous ledger, ledger digest in handoff.json). Not taken: local-reference-links.json (canon resolves bundle citations by namespace and registered sources since phase 3b), and the julien-specific domain brief and final-account format.

10. **Other lineages.** statarb and agQSL are first explored for variants worth reusing (for example agQSL's quota detection, no-start timeout, size preflight and idle watcher, and statarb's recovery diagnoser and implementation stage), then brought under the pathfinder roof as deployments of the canonical engine rather than separate copies.

All questions raised in this note are settled.